

1 - Containerization and application layer load balancing

Name of report: LS_LAB_1_Nikita_Niakhai

Course: DevOps and Security

Performed by Nikita Niakhai 18

Date submission: 27.01.2026

Task 1: Get familiar with Docker Engine

1. Pull Nginx v1.23.3 image from dockerhub registry and confirm it is listed in local images

- I updated packages

```
# sudo apt update && apt upgrade -y
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
```

- I Set up Docker's apt repository [1]

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
```

URIs: <https://download.docker.com/linux/ubuntu>
Suites: noble
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

```
sudo apt update
```

- I installed latest docker

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-  
-buildx-plugin docker-compose-plugin
```

```
nikita@LiveServerUbuntu24:~$ sudo systemctl status docker
■ docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-01-27 19:40:01 UTC; 1min 4s ago
     TriggeredBy: ■ docker.socket
    Docs: https://docs.docker.com
   Main PID: 11293 (dockerd)
      Tasks: 9
    Memory: 27.0M (peak: 27.5M)
       CPU: 1.058s
    CGroup: /system.slice/docker.service
            └─11293 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Jan 27 19:40:00 LiveServerUbuntu24 dockerd[11293]: time="2026-01-27T19:40:00.251019127Z" level=info msg="Restoring contain
Jan 27 19:40:00 LiveServerUbuntu24 dockerd[11293]: time="2026-01-27T19:40:00.313383210Z" level=info msg="Deleting nftables
```

- Pulled nginx

```
nikita@LiveServerUbuntu24:~$ sudo docker pull nginx:1.23.3
1.23.3: Pulling from library/nginx
44cecbfc269: Download complete
1f26f570256: Downloading [=====>] 1 25.17MB/31.41MB
1ff0f94a8007: Download complete
34181e80d10e: Downloading [=====>] 1 24.12MB/25.47MB
d776269cad10: Download complete
e9427fcfa864: Download complete
```

```
nikita@LiveServerUbuntu24:~$ sudo docker image ls | grep nginx
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
nginx:1.23.3 f4e3b6489888 214MB 56.9MB
```

2. Run the pulled Nginx as a container with the below properties
 - a. Map the port to 8080.
 - b. Name the container as `nginx-st18`
 - c. Run it as daemon .

```
root@LiveServerUbuntu24:~# docker run -d --name nginx-st18 -p 8080:80 nginx:1.23.3
5c1f857fd019858d8daaa381cd3047a8e5a30f7945a8d033ef17f19e830f2303
```

3. Confirm port mapping.

- List open ports in host machine. Verified running status in proccesses:

```
root@LiveServerUbuntu24:~# netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.54:53          0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:8080           0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53          0.0.0.0:*               LISTEN
tcp6       0      0 :::8080                 :::*                     LISTEN
udp        0      0 127.0.0.54:53          0.0.0.0:*               *
udp        0      0 127.0.0.53:53          0.0.0.0:*               *
udp        0      0 10.0.2.15:68           0.0.0.0:*               *
```

- I accessed docker and updated packages there and installed `net-tools`. List open ports inside the running container.

```
root@5c1f857fd019:~# apt update && apt upgrade
Get:1 http://deb.debian.org/debian bullseye InRelease [75.1 kB]
Get:2 http://deb.debian.org/debian-security bullseye-security InRelease [27.2 kB]
Get:3 http://deb.debian.org/debian bullseye-updates InRelease [44.0 kB]
Get:4 http://deb.debian.org/debian bullseye/main amd64 Packages [8066 kB]
35% [4 Packages 2208 kB/8066 kB 27%]
```

```
root@5c1f857fd019:~# netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:80             0.0.0.0:*               LISTEN
tcp6       0      0 :::80                   :::*                     LISTEN
```

- I exited from docker CLI and accessed the page. And saw default nginx page

```
# With verbose output (shows headers)
curl -v http://localhost:8080
```

```

root@5c1f857fd019:/# exit
exit
root@LiveServerUbuntu24:~# curl -v http://localhost:8080
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET / HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Server: nginx/1.23.3
< Date: Tue, 27 Jan 2026 19:51:19 GMT
< Content-Type: text/html
< Content-Length: 615
< Last-Modified: Tue, 13 Dec 2022 15:53:53 GMT
< Connection: keep-alive
< ETag: "6398a011-267"
< Accept-Ranges: bytes
<
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

4. Create a Dockerfile similar to the below properties (let's call it container A).
 - Image tag should be Nginx v1.23.3.
 - Create a custom index.html file and copy it to your docker image to replace the Nginx default web page.
 - Build the image from the Dockerfile, tag it during build as `nginx:st18`, check/validate local images, and run your custom made docker image.
 - Stopped prev container to be able to access port 8080 and runned containerA

```

root@LiveServerUbuntu24:~# echo '<h1>st18</h1>' > index.html
root@LiveServerUbuntu24:~# echo -e 'FROM nginx:1.23.3\n\n' > Dockerfile
root@LiveServerUbuntu24:~# echo -e 'COPY index.html /usr/share/nginx/html/index.html\n\n' >> Dockerfile

b70021cd1f000ba8db62260cfa0439d995dc36df7d39831801cc363023e134787
root@LiveServerUbuntu24:~# docker build -t nginx:st18 .

root@LiveServerUbuntu24:~# docker run -d --name containerA -p8080:80 nginx:st18
bf8821cdfd66ba8db62260cfa0439d995dc36df7d39831801cc363023e134787

```

- Access via browser and validate that your custom page is hosted.

```

root@LiveServerUbuntu24:~# curl http://localhost:8080
<h1>st18</h1>

```

Task 2: Work with multi-container environment

1. Create another Dockerfile similar to step 1.4 (Let's call it container B), and an index.html with different content.

```

root@LiveServerUbuntu24:~# tree
├── containerA
│   ├── Dockerfile
│   └── index.html
└── containerB
    ├── Dockerfile
    └── index.html

```

```

root@LiveServerUbuntu24:~# cat ~/containerB/index.html
<h1>Hi bro, I am from different container B image! ST18</h1>
root@LiveServerUbuntu24:~# cat ~/containerB/Dockerfile
FROM nginx:1.23.3

COPY index.html /usr/share/nginx/html/index.html

```

- Created
2. Write a docker-compose file with the below properties (Multi-build: Builds both Dockerfiles and runs both images; Port mapping: Container A should listen to port 8080 and container B should listen to port 9090. (They host two different web pages)
- I wrote it

```

GNU nano 7.2                                docker-compose.yaml *
services:
  a:
    container_name: containerA
    build: containerA
    ports: [ "8080:80" ]
  b:
    container_name: containerB
    build: containerB
    ports: [ "9090:80" ]

```

- I run docker images

```
root@LiveServerUbuntu24:~# docker compose build && docker compose up -d
[+] Building 2.1s (15/15) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 823B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 100B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 100B
=> [internal] load metadata for docker.io/library/nginx:1.23.3
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 31B
=> [1/2] FROM docker.io/library/nginx:1.23.3@sha256:f4e3b6489888647ce1834b601c6c06b9f8c03dee6e097e13ed3e28c01ea
=> => resolve docker.io/library/nginx:1.23.3@sha256:f4e3b6489888647ce1834b601c6c06b9f8c03dee6e097e13ed3e28c01ea3a
=> [internal] load build context
=> => transferring context: 31B
=> CACHED [2/2] COPY index.html /usr/share/nginx/html/index.html
=> CACHED [2/2] COPY index.html /usr/share/nginx/html/index.html
=> [b] exporting to image
=> => exporting layers
=> => exporting manifest sha256:ccc5e20b8a68126b0948624f3c7a59e9c9a4823b1c21adddd200a3690010f29d
=> => exporting config sha256:c55af71aa530348aa72592c7c5086484a47d639893bcc301846218d20acac225
=> => exporting attestation manifest sha256:50b16f12af86e1031ad88f10f0b6c6ed65d8f70e3af965577e78968c7b86167
=> => exporting manifest list sha256:3edac767185637efa71f157a98687e6cf9c0711b68fa1e58d02357be8a8e316d
=> => naming to docker.io/library/root-b:latest
=> => unpacking to docker.io/library/root-b:latest
=> [a] exporting to image
=> => exporting layers
=> => exporting manifest sha256:48ceb99135244cc8e6b1c2c2bf3d08cab5b28338d62cc7cf76adc4b24cf564cb
=> => exporting config sha256:9bd89c36f90dd426612458928463eb2933b8e824fe4f931ea896ad4ec6bf142f
=> => exporting attestation manifest sha256:5059bc945983fd753d319c59a3d036c4087e009b88d7adce109f8135314bb23b
=> => exporting manifest list sha256:bfa74afe408c5de440039043748168c9eb2394b10147f54d3607adf5402feb
=> => naming to docker.io/library/root-a:latest
=> => unpacking to docker.io/library/root-a:latest
=> [b] resolving provenance for metadata file
=> [a] resolving provenance for metadata file
[+] build 2/2
  Image root-a Built
  Image root-b Built
[+] up 3/3
  Container containerA Created
  Container containerB Removed
  Container f6f2c143ce5d_containerB Recreated
```

- I confirm both websites are accessible

```
root@LiveServerUbuntu24:~# curl http://localhost:8080
<h1>st18</h1>
root@LiveServerUbuntu24:~# curl http://localhost:9090
<h1>Hi bro, I am from different container B image! ST18</h1>
root@LiveServerUbuntu24:~#
```

- Volumes: Mount (bind) a directory from the host file system to Nginx containers and update the contents of index.html in the host file system, re-deploy and confirm in the browser that the web page's content is updated.

```

root@LiveServerUbuntu24:~# tree
.
├── containerA
│   ├── Dockerfile
│   └── index.html
├── containerB
│   ├── Dockerfile
│   └── index.html
├── docker-compose.yaml
├── mountable_folder
│   └── index.html
├── mountable_folder_crack
│   └── index.html
└── vboxpostinstall.sh

5 directories, 8 files
root@LiveServerUbuntu24:~# cat mountable_folder_crack/index.html
<h1>This is true root files! I love cakes st18</h1>
root@LiveServerUbuntu24:~# cat mountable_folder/index.html
<h1>This is fake root files! 1 st18</h1>
root@LiveServerUbuntu24:~# cat docker-compose.yaml
services:
  a:
    container_name: containerA
    build: containerA
    ports: [ "8080:80" ]
    volumes: [ "../mountable_folder:/usr/share/nginx/html/" ]
  b:
    container_name: containerB
    build: containerB
    ports: [ "9090:80" ]
    volumes: [ "../mountable_folder_crack:/usr/share/nginx/html/" ]
root@LiveServerUbuntu24:~#

```

```

root@LiveServerUbuntu24:~# docker compose build -q && docker compose up -d
[+] build 2/2
  Image root-a Built 3.0s
  Image root-b Built 3.0s
[+] up 2/2
  Container containerB Recreated
  Container containerA Recreated
root@LiveServerUbuntu24:~# curl localhost:8080
<h1>This is fake root files! 1 st18</h1>
root@LiveServerUbuntu24:~# curl localhost:9090
<h1>This is true root files! I love cakes st18</h1>

```

3. Configure L7 Loadbalaner

- Install Nginx in the host machine or add a third container in the docker-compose that will act as loadbalancer, and configure it in front of two containers in a manner that it should distribute the load in a Weighted Round Robin approach.
- Added configuration for docker-compose

```

GNU nano 7.2                                     docker-compose.yml
services:
  a:
    container_name: containerA
    build: containerA
    ports: [ "8080:80" ]
    volumes: [ "./mountable_folder:/usr/share/nginx/html/" ]
  b:
    container_name: containerB
    build: containerB
    ports: [ "9090:80" ]
    volumes: [ "./mountable_folder_crack:/usr/share/nginx/html/" ]
  lb:
    container_name: lb
    image: nginx:1.23.3
    ports: [ "80:80" ]
    depends_on: [ "a", "b" ]
    volumes: [ "./nginx.conf:/etc/nginx/nginx.conf" ]_

```

- Added nginx for load balancing

```

GNU nano 7.2                                     nginx.conf
worker_processes auto;

events {
    worker_connections 1024;
}

http {
    upstream backend {
        server a:80 weight=3;
        server b:80 weight=1;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://backend;
        }
    }
}

```

- Entered lb container, updated packages and installed nginx
- Updated container and created new

```

root@LiveServerUbuntu24:~# tree
.
├── containerA
│   ├── Dockerfile
│   └── index.html
├── containerB
│   ├── Dockerfile
│   └── index.html
├── docker-compose.yml
├── mountable_folder
│   └── index.html
├── mountable_folder_crack
│   └── index.html
├── nginx.conf
└── vboxpostinstall.sh

5 directories, 9 files
root@LiveServerUbuntu24:~# docker compose build -q && docker compose up -d
[+] build 2/2
  Image root-a Built                               2.7s
  Image root-b Built                               2.7s
[+] up 3/3
  Container containerA Recreated
  Container containerB Recreated
  Container lb Created

```


- Made 10 curls and saw that weights for load balancing divided correctly.
Accessed the page of Nginx ALB and validated, it is load-balancing the traffic.

```
root@LiveServerUbuntu24:~# for i in {1..10}; do curl -s http://localhost > results_curl_lb.txt; done;
root@LiveServerUbuntu24:~# ls
containerA  containerB  docker-compose.yaml  mountable_folder  mountable_folder_crack  nginx.conf  results_curl_lb.txt
root@LiveServerUbuntu24:~# grep -c "containerA" results_curl_lb.txt
7
root@LiveServerUbuntu24:~# grep -c "containerB" results_curl_lb.txt
3
```